

Реализация семантического индекса

что было выбрано?

KD-tree – это бинарное дерево пространственного разбиения, предназначенное для хранения точек в многомерном пространстве.

Каждый узел дерева содержит:

- embedding-вектор
- id объекта
- payload с текстом
- ось разбиения

Почему был выбран?

KD-tree хорошо подходит для задачи nearest neighbor search:

- поддерживает поиск ближайших соседей
- уменьшает количество проверяемых точек
- использует пространственное разбиение
- позволяет выполнять pruning ветвей

В отличие от линейного поиска, дерево не обязано посещать все элементы.



Принцип работы KD-tree

При построении дерева:

1. выбирается ось разбиения
2. точки сортируются по этой оси
3. выбирается медиана
4. медиана становится текущим узлом
5. левая часть уходит в левое поддерево
6. правая часть – в правое.

для балансировки использовалось построение через `median split`.

Неожиданное открытие

Первоначально дерево строилось через последовательные `insert` операции. Но такая версия дерева работала дольше `linear_search`.

Поэтому была реализована `balanced`-версия дерева:

- сначала все точки сохранялись в массив
- затем дерево строилось рекурсивно через медианное разбиение.

Это позволило:

- уменьшить глубину дерева
- улучшить `pruning`
- снизить количество посещаемых узлов.



Поиск ближайших соседей

Во время поиска:

1. дерево спускается в наиболее вероятную ветвь
2. вычисляется расстояние до текущего узла
3. поддерживается heap из top-k лучших результатов
4. проверяется возможность pruning.



Косинусное расстояние

Для сравнения embedding-векторов использовалось косинусное расстояние:

$$\cos \theta = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \cdot \|\vec{b}\|}$$

Перед поиском все векторы нормализовались.



Pruning – это отсечение ветвей, которые гарантированно не содержат более близких соседей.

Проверяется: $\text{abs}(\text{diff}) < \text{worst_distance}$
где:

- diff – расстояние до разделяющей плоскости;
- worst_distance – худший из текущих top-k результатов.

Если плоскость дальше, чем уже найденный сосед, ветвь можно не посещать.

Разделяющая плоскость

Каждый узел дерева задаёт разделяющую гиперплоскость. Циклично разбиваем оси по координатам.

Плоскость делит пространство на две части:

- left subtree;
- right subtree.

Сложность поиска

Теоретическая сложность KD-tree:

$$O(k \cdot \log N)$$

где:

- N – число элементов
- k – число ближайших соседей.

Это достигается благодаря pruning.

Что происходит при высокой размерности

В данной работе embeddings имели размерность: 384
Для KD-tree это является высокой размерностью.

Возникает проблема:

В пространствах большой размерности:

- точки становятся почти равноудалёнными
- pruning начинает работать хуже
- дерево посещает всё больше узлов.

В результате KD-tree постепенно деградирует до линейного поиска.



БЕНЧМАРК

Линейный поиск:

- полный перебор всех векторов;
- вычисление cosine distance для каждого элемента.

KD-tree:

- поиск через пространственное разбиение.

Тестирование проводилось для:

$N \in \{100, 500, 1000, 5000, 10000\}$

Для всех тестов:

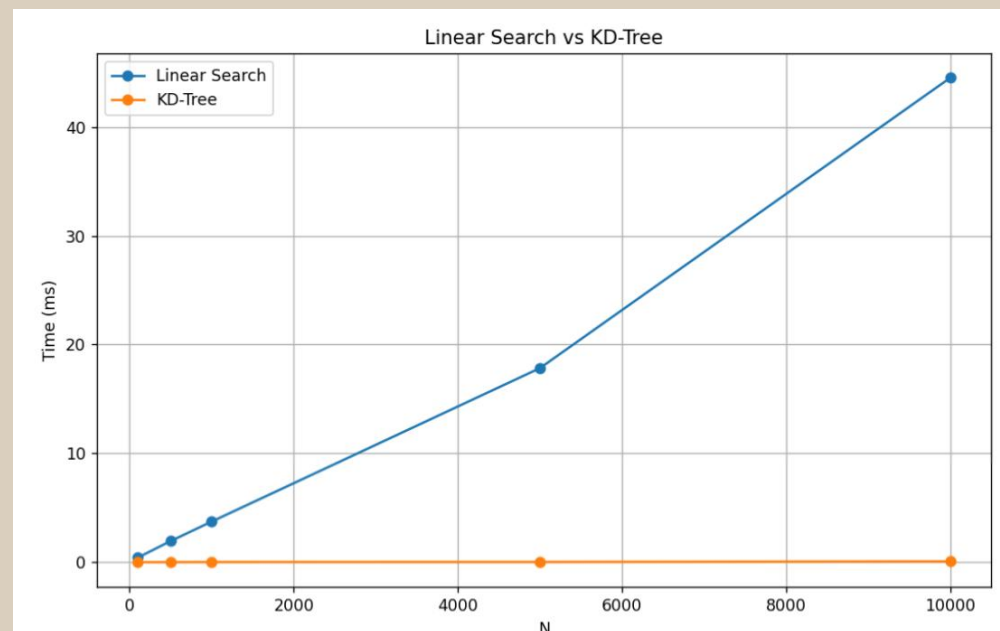
- использовались одинаковые embeddings
- запрос: "космический аппарат и его траектория"

На графике видно, что для $N \geq 100$:

- линейный поиск растёт почти линейно;
- KD-Tree растёт значительно медленнее.

Final Results:

N	Linear (ms)	KDTree (ms)	Speedup
100	0.43	0.01	36.29
500	1.95	0.02	110.76
1000	3.74	0.03	126.83
5000	17.84	0.04	509.79
10000	44.52	0.09	504.78



ВЫВОД:

Что удивило

Изначально обычное KD-Tree работало медленнее линейного поиска.

Причина:

- дерево было несбалансированным
- pruning почти не работал
- размерность эмбедингов слишком высокая.

После реализации Balanced KD-Tree ситуация изменилась полностью.

Что не сработало так, как ожидалось

Теоретическая сложность $O(k \cdot \log N)$ не всегда достигается. Для эмбедингов высокой размерности KD-Tree постепенно теряет эффективность.

Почему проклятие размерности – реальная проблема

При размерности 384:

- расстояния между точками становятся похожим
 - разделение пространства работает хуже
 - pruning перестаёт эффективно отсекал ветви.
- Из-за этого KD-Tree плохо масштабируется на очень больших эмбедингах.